MTRC

MODERN INSTITUTE OF TECHNOLOGY AND RESEARCH CENTRE

Name of Faculty: Gendal Lal

Subject & Subject Code: Theory of Computation & 4CSE4-03, 4AM4-03, 4AD4-03

Branch: CSE, AIML and AI & DS Batch:

Semester: IV **Session:** 2023-24(Even Sem.)

Unit No.: 3 **Date of Submission: 11/03/2024**

Lect.No.	Topics Covered	Page No.	Total pages
1	PDA definition	1-7	7
2	PDA acceptability	8-12	5
3	NPDA	13-14	2
4	PDA and CFG	15-16	2
5	CFG to PDA	17-18	2
6	Pumping lemma for CFL	19-21	3

References Book:

1. "Introduction to the Theory of Computation "by Micheal Sipser

2. "Introduction to computability "by Fred C Henie

FACULTY **HOD** **DEAN ACADEMICS**

UNIT-3 (TOC)

LECTURE NO: 1

PDA: Definition

Basic Structure of PDA

Pushdown automata is a way to implement a CFG in the same way we design DFA for a regular grammar. A DFA can remember a finite amount of information, but a PDA can remember an infinite amount of information.

·Pushdown automata is simply an NFA augmented with an "external stack memory". The addition of stack is used to provide a last-in-first-out memory management capability to Pushdown automata. Pushdown automata can store an unbounded amount of information on the stack. It can access a limited amount of information on the stack. A PDA can push an element onto the top of the stack and pop off an element from the top of the stack. To read an element into the stack, the top elements must be popped off and are lost.

·A PDA is more powerful than FA. Any language which can be acceptable by FA can also be acceptable by PDA. PDA also accepts a class of language which even cannot be accepted by FA. Thus PDA is much more superior to FA.

Basically a pushdown automaton is –

"Finite state machine" + "a stack"

MTRC

A pushdown automaton has three components –

1. Input tape: The input tape is divided in many cells or symbols. The input head is read-only and may only move from left to right, one symbol at a time.

2. Finite control: The finite control has some pointer which points the current symbol which is to be read.

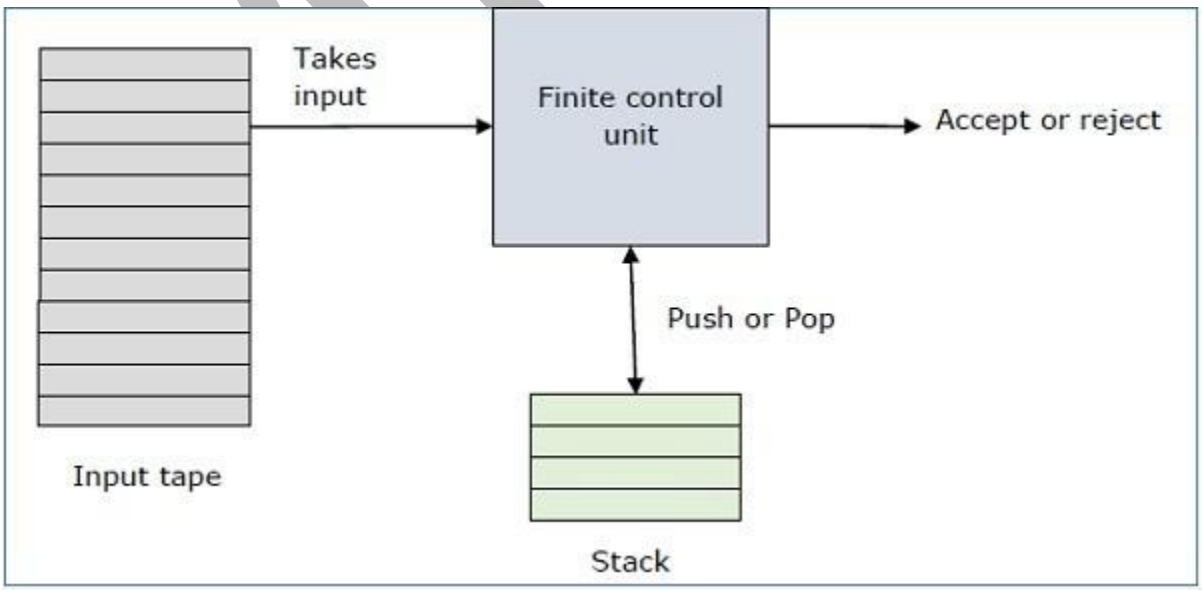
3. Stack: The stack is a structure in which we can push and remove the items from one end only. It has an infinite size. In PDA, the stack is used to store the items temporarily.

The stack head scans the top symbol of the stack.

A stack does two operations –

- **Push** – a new symbol is added at the top.
- **Pop** – the top symbol is read and removed.

A PDA may or may not read an input symbol, but it has to read the top of the stack in every transition.

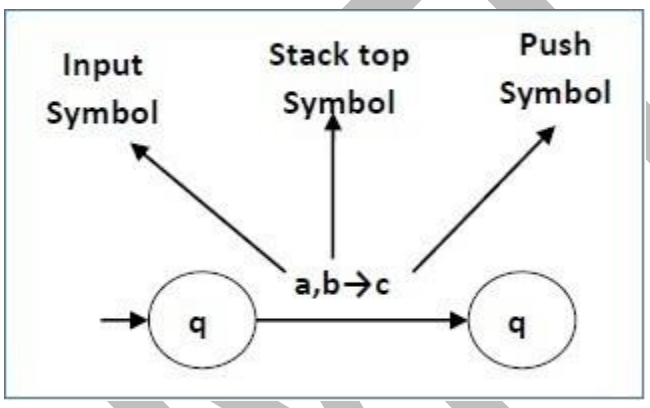


ATRC

A PDA can be formally described as a 7-tuple **(Q, ∑, S, δ, q0, Zo, F)**

- Q is the finite number of states
- ∑ is input alphabet
- S is stack symbols
- δ is the transition function: Q × (∑ ∪ {}) × S × Q × S*
- q0 is the initial state (q0 ∈ Q)
- Zo is the initial stack top symbol (Zo ∈ S)
- F is a set of accepting states (F ∈ Q)

The following diagram shows a transition in a PDA from a state q1 to state q2, labeled as a,b → c –



This means at state **q1**, if we encounter an input string '**a**' and top symbol of the stack is '**b**', then we pop '**b**', push '**c**' on top of the stack and move to state **q2**.

Terminologies Related to PDA

Instantaneous Description

Instantaneous Description (ID) is an informal notation of how a PDA "computes" a input string and make a decision that string is accepted or rejected.

A ID is a triple (q, w, α), where:
1. q is the current state.
2. w is the remaining input.
3. α, where: is the stack contents, top at the left.

OR

The instantaneous description (ID) of a PDA is represented by a triplet (q, w, s) where

- q is the state
- w is unconsumed input
- s is the stack contents

Turnstile Notation

The "turnstile" notation is used for connecting pairs of ID's that represent one or many moves of a PDA. The process of transition is denoted by the turnstile symbol "⊢".

Consider a PDA (Q, ∑, S, δ, q0, I, F). A transition can be mathematically represented by the following turnstile notation –

(p, aw, Tβ) ⊢ (q, w, αβ)

This implies that while taking a transition from state **p** to state **q**, the input symbol '**a**' is consumed, and the top of the stack '**T**' is replaced by a new string '**α**''.

MTRC

Example : Define the pushdown automata for language {anbn | n > 0}

Solution : M = where Q = {q0, q1 } and Σ = {a, b } and Γ = {A, Z } and &delta is given by :

&delta(q0, a, Z) = { (q0, AZ) }

&delta(q0, a, A) = { (q0, AA) }

&delta(q0, b, A) = { (q1, ∈) }

&delta(q1, b, A) = { (q1, ∈) }

&delta(q1, ∈, Z) = { (q1, ∈) }

Let us see how this automata works for aaabbb.

Row	State	Input	δ(transition function used)	Stack(Leftmost symbol represents top of stack)	State after move
1	q0	aaabbb		Z	q0
2	q0	**a**aabbb	δ(q0,a,Z)={(q0,AZ)}	AZ	q0
3	q0	a**a**abbb	δ(q0,a,A)={(q0,AA)}	AAZ	q0
4	q0	aa**a**bbb	δ(q0,a,A)={(q0,AA)}	AAAZ	q0
5	q0	aaa**b**bb	δ(q0,b,A)={(q1,∈)}	AAZ	q1
6	q1	aaab**b**b	δ(q1,b,A)= {(q1,∈)}	AZ	q1
7	q1	aaabb**b**	δ(q1,b,A)= {(q1,∈)}	Z	q1
8	q1	∈	δ(q1, ∈,Z)= {(q1,∈)}	∈	q1

Explanation : Initially, the state of automata is q0 and symbol on stack is Z and the input is aaabbb as shown in row 1. On reading 'a' (shown in bold in row 2), the state will remain q0 and it will

push symbol A on stack. On next 'a' (shown in row 3), it will push another symbol A on stack. After reading 3 a's, the stack will be AAAZ with A on the top. After reading 'b' (as shown in row 5), it will pop A and move to state q1 and stack will be AAZ. When all b's are read, the state will be q1 and stack will be Z. In row 8, on input symbol '∈' and Z on stack, it will pop Z and stack will be empty. This type of acceptance is known as acceptance by empty stack.

Note :

- The above pushdown automaton is deterministic in nature because there is only one move from a state on an input symbol and stack symbol.
- The non-deterministic pushdown automata can have more than one move from a state on an input symbol and stack symbol.
- It is not always possible to convert non-deterministic pushdown automata to deterministic pushdown automata.
- Expressive Power of non-deterministic PDA is more as compared to expressive deterministic PDA as some languages which are accepted by NPDA but not by deterministic PDA which will be discussed in next article.
- The push down automata can either be implemented using accepetance by empty stack or accepetance by final state and one can be converted to another.

Question : Which of the following pairs have DIFFERENT expressive power?

MTRC

A. Deterministic finite automata(DFA) and Non-deterministic finite automata(NFA)
B. Deterministic push down automata(DPDA)and Non-deterministic push down automata(NPDA)
C. Deterministic single-tape Turing machine and Non-deterministic single-tape Turing machine
D. Single-tape Turing machine and multi-tape Turing machine

Solution : Every NFA can be converted into DFA. So, there expressive power is same. As discussed above, every NPDA can't be converted to DPDA. So, the power of NPDA and DPDA is not same. Hence option (B) is correct.

Note – If we want zero or more moves of a PDA, we have to use the symbol ($\vdash^*$) for it.

MITRC

LECTURE NO:-2

PDA acceptabilitY

There are two different ways to define PDA acceptability.

1. Final State Acceptability

In final state acceptability, a PDA accepts a string when, after reading the entire string, the PDA is in a final state. From the starting state, we can make moves that end up in a final state with any stack values. The stack values are irrelevant as long as we end up in a final state.

For a PDA (Q, ∑, S, δ, q0, I, F), the language accepted by the set of final states F is –

L(PDA) = {w | (q0, w, I) ⊢* (q, ε, x), q ∈ F}

for any input stack string **x**.

Empty Stack Acceptability

Here a PDA accepts a string when, after reading the entire string, the PDA has emptied its stack.

For a PDA (Q, ∑, S, δ, q0, I, F), the language accepted by the empty stack is –

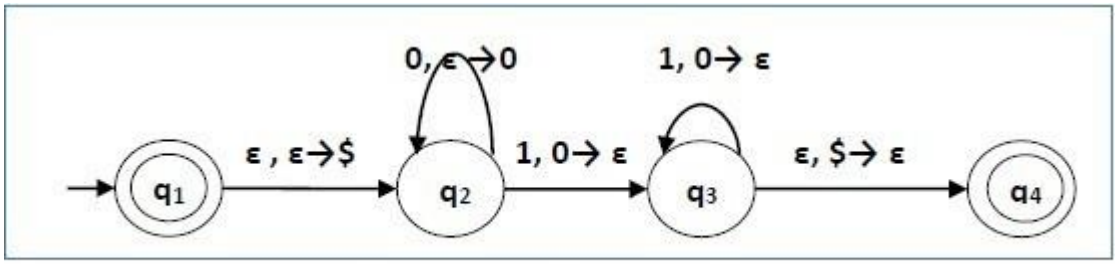
L(PDA) = {w | (q0, w, I) ⊢* (q, ε, ε), q ∈ Q}

Example:

Construct a PDA that accepts **L = {0n 1n | n ≥ 0}**

MITRC

Solution



PDA for L= {$0^n 1^n$ | n≥0}

This language accepts L = {ε, 01, 0011, 000111,.....}

Here, in this example, the number of **'a'** and **'b'** have to be same.

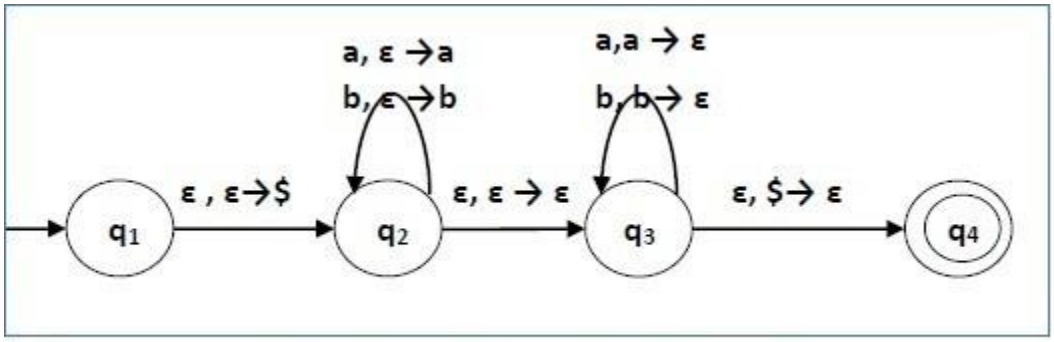
- Initially we put a special symbol '$' into the empty stack.
- Then at state q2, if we encounter input 0 and top is Null, we push 0 into stack. This may iterate. And if we encounter input 1 and top is 0, we pop this 0.
- Then at state q3, if we encounter input 1 and top is 0, we pop this 0. This may also iterate. And if we encounter input 1 and top is 0, we pop the top element.
- If the special symbol '$' is encountered at top of the stack, it is popped out and it finally goes to the accepting state q4.

Example

Construct a PDA that accepts L = {ε}xxQx* wwR | w = (a+b)* }

Solution

MTRC



PDA for L= {ww^R | w = (a+b)*}

Initially we put a special symbol '$' into the empty stack. At state **q2**, the **w** is being read. In state **q3**, each 0 or 1 is popped when it matches the input. If any other input is given, the PDA will go to a dead state. When we reach that special symbol '$', we go to the accepting state **q4**.

Equivalence of Acceptance by Final State and Empty Stack

If L = N(P1) for some PDA P1, then there is a PDA P2 such that L = L(P2). That means the language accepted by empty stack PDA will also be accepted by final state PDA.

If there is a language L = L (P1) for some PDA P1 then there is a PDA P2 such that L = N(P2). That means language accepted by final state PDA is also acceptable by empty stack PDA.

Example:

Construct a PDA that accepts the language L over {0, 1} by empty stack which accepts all the string of 0's and 1's in which a number of 0's are twice of number of 1's.

MTRC

Solution:

There are two parts for designing this PDA:

If 1 comes before any 0's

If 0 comes before any 1's.

We are going to design the first part i.e. 1 comes before 0's. The logic is that read single 1 and push two 1's onto the stack. Thereafter on reading two 0's, POP two 1's from the stack. The δ can be

δ(q0, 1, Z) = (q0, 11, Z) Here Z represents that stack is empty

δ(q0, 0, 1) = (q0, ∧)

Now, consider the second part i.e. if 0 comes before 1's. The logic is that read first 0, push it onto the stack and change state from q0 to q1. [Note that state q1 indicates that first 0 is read and still second 0 has yet to read].

Being in q1, if 1 is encountered then POP 0. Being in q1, if 0 is read then simply read that second 0 and move ahead. The δ will be:

δ(q0, 0, Z) = (q1, 0Z)

δ(q1, 0, 0) = (q1, 0)

δ(q1, 0, Z) = (q0, ∧) (indicate that one 0 and one 1 is already read, so simply read the second 0)

δ(q1, 1, 0) = (q1, ∧)

MTRC

Now, summarize the complete PDA for given L is:

$\delta(q0, 1, Z) = (q0, 11Z)$

$\delta(q0, 0, 1) = (q1, \wedge)$

$\delta(q0, 0, Z) = (q1, 0Z)$

$\delta(q1, 0, 0) = (q1, 0)$

$\delta(q1, 0, Z) = (q0, \wedge)$

$\delta(q0, \wedge, Z) = (q0, \wedge)$ ACCEPT state

MITRC

LECTURE NO:-3

NPDA

Non-deterministic Pushdown Automata

The non-deterministic pushdown automata is very much similar to NFA. We will discuss some CFGs which accepts NPDA.

The CFG which accepts deterministic PDA accepts non-deterministic PDAs as well. Similarly, there are some CFGs which can be accepted only by NPDA and not by DPDA. Thus NPDA is more powerful than DPDA.

Example:

Design PDA for Palindrome strips.

Solution:

Suppose the language consists of string L = {ba, aa, bb, bab, bbabb, aabaa, ...]. The string can be odd palindrome or even palindrome. The logic for constructing PDA is that we will push a symbol onto the stack till half of the string then we will read each symbol and then perform the pop operation. We will compare to see whether the symbol which is popped is similar to the symbol which is read. Whether we reach to end of the input, we expect the stack to be empty.

This PDA is a non-deterministic PDA because finding the mid for the given string and reading the string from left and matching it with from right (reverse) direction leads to non-deterministic moves. Here is the ID.

MITRC

1. $\delta(q1, a, Z) = (q1, aZ)$
2. $\delta(q0, b, Z) = (q1, bZ)$
3. $\delta(q0, a, a) = (q1, aa)$
4. $\delta(q1, a, b) = (q1, ab)$
5. $\delta(q1, a, b) = (q1, ba)$
6. $\delta(q1, b, b) = (q1, bb)$

Pushing the symbols onto the stack

7. $\delta(q1, a, a) = (q2, \varepsilon)$
8. $\delta(q1, b, b) = (q2, \varepsilon)$
9. $\delta(q2, a, a) = (q2, \varepsilon)$
10. $\delta(q2, b, b) = (q2, \varepsilon)$
11. $\delta(q2, \varepsilon, Z) = (q2, \varepsilon)$

Popping the symbols on reading the same kind of symbol

Simulation of abaaba

1. $\delta(q1, abaaba, Z)$ Apply rule 1
2. $\vdash \delta(q1, baaba, aZ)$ Apply rule 5
3. $\vdash \delta(q1, aaba, baZ)$ Apply rule 4
4. $\vdash \delta(q1, aba, abaZ)$ Apply rule 7
5. $\vdash \delta(q2, ba, baZ)$ Apply rule 8
6. $\vdash \delta(q2, a, aZ)$ Apply rule 7
7. $\vdash \delta(q2, \varepsilon, Z)$ Apply rule 11
8. $\vdash \delta(q2, \varepsilon)$ Accept

LECTURE NO:-4

PDA And CFG

If a grammar **G** is context-free, we can build an equivalent nondeterministic PDA which accepts the language that is produced by the context-free grammar **G**. A parser can be built for the grammar **G**.

Also, if **P** is a pushdown automaton, an equivalent context-free grammar G can be constructed where

L(G) = L(P)

In the next two topics, we will discuss how to convert from PDA to CFG and vice versa.

Algorithm to find PDA corresponding to a given CFG

Input − A CFG, G = (V, T, P, S)

Output − Equivalent PDA, P = (Q, ∑, S, δ, q0, I, F)

Step 1 − Convert the productions of the CFG into GNF.

Step 2 − The PDA will have only one state {q}.

Step 3 − The start symbol of CFG will be the start symbol in the PDA.

Step 4 − All non-terminals of the CFG will be the stack symbols of the PDA and all the terminals of the CFG will be the input symbols of the PDA.

MTRC

Step 5 – For each production in the form **A → aX** where a is terminal and **A, X** are combination of terminal and non-terminals, make a transition **δ (q, a, A)**.

Problem

Construct a PDA from the following CFG.

G = ({S, X}, {a, b}, P, S)

where the productions are –

S → XS | ε , A → aXb | Ab | ab

Solution

Let the equivalent PDA,

P = ({}, {, b}, {, b, X, S}, δ, q, S)

where δ –

δ(q, , S) = {(q, XS), (q,)}

δ(q, , X) = {q, aXb), (q, Xb), (q, ab)}

δ(q, a, a) = {(q,)}

δ(q, 1, 1) = {(q,)}

Algorithm to find CFG corresponding to a given PDA

Input – A CFG, G = (V, T, P, S)

Output – Equivalent PDA, P = (Q, ∑, S, δ, q0, I, F) such that the non- terminals of the grammar G will be {Xwx | w,x ∈ Q} and the start state will be Aq0,F.

MITRC

Step 1 – For every w, x, y, z ∈ Q, m ∈ S and a, b ∈ ∑, if δ (w, a,) contains (y, m) and (z, b, m) contains (x,), add the production rule Xwx → a Xyzb in grammar G.

Step 2 – For every w, x, y, z ∈ Q, add the production rule Xwx → XwyXyx in grammar G.

Step 3 – For w ∈ Q, add the production rule Xww → ε}) x S x Q x S* in grammar G.

CFG to PDA Conversion

The first symbol on R.H.S. production must be a terminal symbol. The following steps are used to obtain PDA from CFG is:

Step 1: Convert the given productions of CFG into GNF.

Step 2: The PDA will only have one state {q}.

Step 3: The initial symbol of CFG will be the initial symbol in the PDA.

Step 4: For non-terminal symbol, add the following rule:

1. δ(q, , A) = (q,)

Where the production rule is A → α),where:

Step 5: For each terminal symbols, add the following rule:

1. δ(q, a, a) = (q,) **for** every terminal symbol

Example 1:

Convert the following grammar to a PDA that accepts the same language.

1.S → 0S1 | A

2.A → 1A0 | S | ε}))×S×Q×S*

Solution:

The CFG can be first simplified by eliminating unit productions:

1.S → 0S1 | 1S0 | ε}))×S×Q×S*

Now we will convert this CFG to GNF:

1.S → 0SX | 1SY | ε}))×S×Q×S*

2. X → 1

3. Y → 0

The PDA can be:

R1: δ(q, , S) = (q, 0SX) | (q, 1SY) | (q,)}

R2: δ(q, , X) = q, 1)}

R3: δ(q, , Y) = q, 0)}

R4: δ(q, 0, 0) = q,)}

R5: δ(q, 1, 1) = q,)}

Example 2:

Construct PDA for the given CFG, and test whether 0104 is acceptable by this PDA.

1.S → 0BB

2.B → 0S | 1S | 0

Solution:

The PDA can be given as:

MITRC

1.A = {(q), (0, 1), (S, B, 0, 1), δ, q, S, ?}

The production rule δ can be:

R1: δ(q, , S) = {(q, 0BB)}

R2: δ(q, , B) = {(q, 0S) | (q, 1S) | (q, 0)}

R3: δ(q, 0, 0) = {(q,)}

R4: δ(q, 1, 1) = {(q,)}

Testing 010^4 i.e. 010000 against PDA:

1.δ(q, 010000, S) ⊢ δ(q, 010000, 0BB)

2. ⊢ δ(q, 10000, BB) R1

3. ⊢ δ(q, 10000,1SB) R3

4. ⊢ δ(q, 0000, SB) R2

5. ⊢ δ(q, 0000, 0BBB) R1

6. ⊢ δ(q, 000, BBB) R3

7. ⊢ δ(q, 000, 0BB) R2

8. ⊢ δ(q, 00, BB) R3

9. ⊢ δ(q, 00, 0B) R2

10. ⊢ δ(q, 0, B) R3

11. ⊢ δ(q, 0, 0) R2

12. ⊢ δ(q,) R3

13. ACCEPT

Thus 010^4 is accepted by the PDA.

MITRC